Специфікація програмного забезпечення

В.1 ВСТУП

В.1.1 Огляд продукту

Програмна система управління проєктами є веборієнтованим застосунком, призначеним для організації роботи з проєктами, задачами, учасниками команди, дедлайнами та аналітичною інформацією. Система забезпечує централізоване середовище для створення проєктів, розподілу задач, візуального контролю їх виконання за допомогою Kanban-дошки, календарного планування та перегляду основних показників виконання робіт.

Програмна система складається з клієнтської та серверної частин. Клієнтська частина забезпечує користувацький інтерфейс і взаємодію користувача з функціональними модулями системи. Серверна частина відповідає за обробку запитів, автентифікацію, авторизацію, роботу з базою даних та збереження основної інформації про користувачів, проєкти, колонки й задачі.

В.1.2 Мета

Метою створення програмної системи є забезпечення зручного інструменту для управління проєктами та задачами, підтримки командної взаємодії, контролю дедлайнів і перегляду стану виконання робіт.

Система повинна надати користувачам можливість працювати з проєктами в єдиному середовищі, зменшити розрізненість інформації та спростити процес планування, виконання й моніторингу задач. Додатково система має підвищити прозорість командної роботи завдяки розподілу відповідальності між учасниками та наданню актуальної інформації про стан проєктів.

В.1.3 Межі

У межах програмної системи передбачено реалізацію таких основних можливостей:

- реєстрація та автентифікація користувачів;
- підтвердження електронної пошти користувача;
- рольова модель доступу;
- створення, перегляд, редагування та видалення проєктів;
- керування учасниками проєкту;
- створення та редагування колонок Kanban-дошки;
- створення, перегляд, редагування, переміщення та видалення задач;
- призначення виконавців, дедлайнів і міток задачам;
- календарне відображення задач із дедлайнами;
- перегляд аналітичної інформації;
- редагування профілю користувача;
- перемикання мови інтерфейсу та теми оформлення.

Система не передбачає реалізацію окремого мобільного застосунку. Взаємодія з Google Calendar реалізується через формування посилання для створення події, без повної двосторонньої синхронізації із сервісом.

В.1.4 Посилання

В.1.5 Означення та абревіатури

API – програмний інтерфейс, який забезпечує взаємодію між клієнтською та серверною частинами системи.

JWT – токен, що використовується для автентифікації користувача та доступу до захищених ресурсів.

REST API – архітектурний підхід до побудови API, у якому взаємодія між клієнтом і сервером здійснюється через HTTP-запити.

Frontend – клієнтська частина програмної системи, з якою безпосередньо взаємодіє користувач.

Backend – серверна частина програмної системи, що обробляє запити, виконує бізнес-логіку та працює з базою даних.

Kanban-дошка – спосіб візуального представлення задач у вигляді колонок і карток.

RBAC – модель керування доступом на основі ролей користувачів.

В.2 ЗАГАЛЬНИЙ ОПИС

В.2.1 Перспективи продукту

Програмна система може використовуватися як основа для організації роботи невеликих команд, навчальних проєктів або внутрішніх робочих процесів. Завдяки модульній структурі система може бути розширена в майбутньому, зокрема шляхом додавання коментарів до задач, розширеної аналітики, сповіщень або додаткових інтеграцій із зовнішніми сервісами.

Система є вебзастосунком і може бути розгорнута у середовищі, що підтримує Node.js, Next.js, NestJS та MongoDB.

В.2.2 Функції продукту

Основними функціями програмної системи є:

- створення облікового запису користувача;
- вхід користувача до системи;
- перевірка прав доступу відповідно до ролі;
- перегляд списку проєктів;
- створення, редагування та видалення проєктів;
- додавання учасників до проєкту та зміна їхніх ролей;
- створення колонок Kanban-дошки;
- створення, редагування, переміщення та видалення задач;
- перегляд задач на Kanban-дошці;
- відображення задач із дедлайнами у календарі;
- додавання задачі до Google Calendar;

– перегляд аналітики за задачами та дедлайнами;
– редагування профілю користувача;
– зміна пароля;
– перемикання мови інтерфейсу;
– перемикання теми оформлення.

В.2.3 Характеристики користувачів

У системі передбачено три основні ролі користувачів:
– адміністратор;
– менеджер проєкту;
– розробник.

Адміністратор має розширені можливості керування користувачами та їхніми ролями. Менеджер проєкту може створювати й редагувати проєкти, керувати учасниками, колонками та задачами. Розробник може працювати із задачами в межах доступних йому проєктів відповідно до наданих прав.

Очікується, що користувачі мають базові навички роботи з вебзастосунками та розуміють основні принципи роботи з задачами, дедлайнами й проєктами.

В.2.4 Загальні обмеження

Програмна система повинна працювати у сучасних веббраузерах, які підтримують JavaScript, cookies і localStorage. Для коректної роботи системи необхідне підключення до серверної частини через REST API.

Серверна частина повинна мати доступ до MongoDB для збереження даних. Для авторизації використовується JWT, тому захищені запити повинні містити токен доступу.

Система не призначена для роботи без серверної частини та бази даних у повноцінному режимі. Також вона не є повною заміною корпоративним

системам управління складними проєктами з розширеними workflow, складною звітністю та багаторівневими бізнес-процесами.

В.2.5 Припущення й залежності

Передбачається, що користувач має доступ до інтернету або локальної мережі, у якій розгорнуто систему. Також передбачається, що серверна частина та база даних доступні під час роботи користувача із застосунком.

Робота системи залежить від коректного налаштування змінних середовища, зокрема адреси серверної частини, параметрів підключення до MongoDB, секретного ключа JWT та налаштувань поштового сервісу для підтвердження електронної пошти.

В.3 КОНКРЕТНІ ВИМОГИ

В.3.1 Вимоги до зовнішніх інтерфейсів

В.3.1.1 Інтерфейс користувача

Користувацький інтерфейс повинен забезпечувати доступ до основних модулів системи: авторизації, реєстрації, списку проєктів, Kanban-дошки, календаря, аналітики, профілю користувача та адміністративного розділу.

Інтерфейс повинен бути зрозумілим, адаптивним і підтримувати світлу та темну теми оформлення. Також система повинна підтримувати українську та англійську мови інтерфейсу.

Основні дії користувача мають супроводжуватися повідомленнями про успішне виконання операцій або про помилки.

В.3.1.2 Апаратний інтерфейс

Спеціальні апаратні засоби для роботи системи не потрібні. Користувач взаємодіє із системою через персональний комп’ютер, ноутбук, планшет або смартфон із веббраузером.

В.3.1.3 Програмний інтерфейс

Клієнтська частина повинна взаємодіяти із серверною частиною через REST API. Дані між frontend- та backend-частинами передаються у форматі JSON.

Серверна частина повинна забезпечувати API для роботи з автентифікацією, користувачами, проєктами, колонками, задачами, аналітикою та сповіщеннями.

В.3.1.4 Комунікаційний протокол

Взаємодія між клієнтською та серверною частинами здійснюється через HTTP або HTTPS. Для захищених запитів використовується заголовок Authorization із JWT-токеном.

В.3.1.5 Обмеження пам’яті

Система не встановлює спеціальних обмежень пам’яті на стороні користувача. Для коректної роботи необхідно, щоб браузер мав достатній обсяг пам’яті для завантаження сторінок, компонентів інтерфейсу та збереження частини локальних даних у localStorage.

На серверній стороні обсяг пам’яті залежить від кількості користувачів, проєктів, задач і обсягу даних у базі MongoDB.

В.3.1.6 Операції

Система повинна підтримувати такі основні операції:

- реєстрація користувача;
- авторизація користувача;
- вихід із системи;
- перегляд і редагування профілю;

– створення, редагування та видалення проєкту;
– додавання й видалення учасників проєкту;
– створення, редагування, переміщення та видалення задач;
– створення, редагування та видалення колонок;
– перегляд календаря задач;
– перегляд аналітики;
– керування ролями користувачів адміністратором.

В.3.1.7 Функції продукту

Функції програмного продукту повинні забезпечувати повний цикл роботи з проєктами та задачами: від створення проєкту до контролю виконання робіт і перегляду аналітичної інформації. Система повинна надавати користувачу можливість створювати проєкти, переглядати їх список, редагувати основні дані, видаляти проєкти та керувати учасниками команди.

У межах кожного проєкту система повинна забезпечувати роботу з Kanban-дошкою. Користувач повинен мати можливість створювати колонки, змінювати їх порядок, додавати задачі до відповідних колонок і переміщувати їх між етапами виконання. Це необхідно для візуального контролю стану робіт і швидкого оцінювання прогресу проєкту.

Система повинна дозволяти створювати, переглядати, редагувати та видаляти задачі. Для кожної задачі має бути передбачена можливість вказати назву, опис, дедлайн, виконавця та мітки, а також переглядати детальну інформацію про задачу.

Для підтримки планування система повинна відображати задачі з установленими дедлайнами у календарному вигляді. Користувач повинен мати можливість переглядати задачі за датами, контролювати найближчі терміни виконання та додавати задачі до Google Calendar через сформоване посилання.

Для моніторингу виконання робіт система повинна надавати аналітичну інформацію щодо загальної кількості задач, найближчих дедлайнів,

прострочених задач та інших показників, необхідних для оцінювання стану проєктів.

Окремо система повинна забезпечувати роботу з профілем користувача: перегляд і редагування особистих даних, зміну пароля, вихід із системи, а також налаштування мови інтерфейсу та теми оформлення.

В.3.1.8 Припущення й залежності

Коректна робота функцій продукту залежить від доступності серверної частини, бази даних MongoDB та правильного налаштування змінних середовища. Для роботи з Google Calendar користувач повинен мати доступ до відповідного сервісу Google у браузері.

В.3.2 Властивості програмного продукту

В.3.2.1 Надійність

Система повинна коректно обробляти помилки авторизації, помилки введення даних і помилки взаємодії з серверною частиною. У разі завершення або недійсності сесії користувач повинен бути перенаправлений на сторінку входу.

В.3.2.2 Доступність

Програмна система повинна бути доступною через веббраузер за умови працездатності клієнтської частини, серверної частини та бази даних. Доступ до захищених сторінок повинен надаватися лише авторизованим користувачам.

В.3.2.3 Безпека

Система повинна забезпечувати автентифікацію користувачів, збереження паролів у захищеному вигляді, авторизацію за JWT-токеном і

розмежування доступу на основі ролей. Доступ до адміністративних функцій має бути обмежений для користувачів без відповідної ролі.

В.3.2.4 Супроводжуваність

Програмний код повинен мати модульну структуру. Клієнтська частина має бути розділена на сторінки, компоненти, контексти, утиліти та API-шар. Серверна частина має бути організована за модулями NestJS: автентифікація, користувачі, проєкти, колонки, задачі, аналітика та сповіщення.

В.3.2.5 Переносимість

Система повинна мати можливість запуску в середовищі, що підтримує Node.js. Адреси серверної частини, бази даних та інші параметри повинні задаватися через змінні середовища, що дозволяє переносити систему між локальним, тестовим і production-середовищем.

В.3.2.6 Продуктивність

Система повинна забезпечувати швидке завантаження сторінок і швидку реакцію інтерфейсу на дії користувача. Операції створення, редагування та переміщення задач мають виконуватися без помітних затримок для користувача.

В.3.3 Атрибути програмного продукту

До основних атрибутів програмного продукту належать:

- модульність;
- адаптивність;
- підтримка ролей користувачів;
- підтримка багатомовного інтерфейсу;
- підтримка світлої та темної тем;

– інтеграція з REST API;
– можливість подальшого розширення функціональності.

Система повинна бути зручною для користувачів, які працюють із проєктами та задачами, і достатньо гнучкою для подальшого розвитку.

В.3.4 Вимоги бази даних

Для збереження даних використовується MongoDB. Взаємодія серверної частини з базою даних здійснюється за допомогою Mongoose.

У базі даних повинні зберігатися такі основні сутності:

– користувачі;
– токени верифікації;
– проєкти;
– учасники проєктів;
– колонки Kanban-дошки;
– задачі;
– сповіщення.

Для користувача зберігаються електронна пошта, пароль, повне ім’я, роль, аватар, статус верифікації та службові дати. Для проєкту зберігаються назва, опис, власник, учасники та порядок колонок. Для задачі зберігаються назва, опис, статус, колонка, проєкт, виконавець, дедлайн і мітки.

Зв’язки між сутностями повинні забезпечувати належність задач до проєктів і колонок, а також належність користувачів до проєктів.

В.3.5 Інші вимоги

Програмна система повинна бути реалізована з використанням таких технологій:

– frontend – Next.js, React, TypeScript, CSS Modules;
– backend – NestJS, TypeScript;
– база даних – MongoDB, Mongoose;

– автентифікація – JWT, Passport;
– взаємодія з API – Axios;
– робота з датами – date-fns;
– drag-and-drop – @hello-pangea/dnd.

В.4 ДОДАТКОВІ МАТЕРІАЛИ

В.4.1 Діаграма компонентів (Component Diagram)

Діаграма компонентів (див. рис. В.1) ілюструє високорівневу архітектуру програмної системи управління проєктами та демонструє взаємозв'язки між її основними модулями. Вона показує, як окремі компоненти взаємодіють між собою для забезпечення роботи користувацького інтерфейсу, обробки запитів і збереження даних.

Архітектура системи відповідає клієнт-серверній моделі та складається з frontend-застосунку, backend-сервера, REST API та бази даних MongoDB. Клієнтська частина, розроблена з використанням React, Next.js та TypeScript, забезпечує графічний інтерфейс користувача і відповідає за відображення проєктів, задач, Kanban-дошки, календаря, аналітики та профілю користувача.

Backend-частина, реалізована з використанням NestJS, містить основну бізнес-логіку системи. Вона обробляє запити, виконує автентифікацію та авторизацію користувачів, перевіряє права доступу, а також забезпечує виконання операцій із проєктами, колонками, задачами та користувачами.

Взаємодія між клієнтською та серверною частинами здійснюється через REST API. Дані передаються у форматі JSON, що забезпечує стандартизований обмін інформацією між компонентами системи.

Для постійного зберігання даних використовується MongoDB. Backend взаємодіє з базою даних за допомогою Mongoose, що забезпечує моделювання даних, перевірку структури документів та виконання операцій з базою даних.

Поділ системи на окремі компоненти підвищує зручність супроводу, масштабованість і надійність програмного забезпечення. Кожен компонент

може змінюватися або розширюватися з мінімальним впливом на інші частини системи.

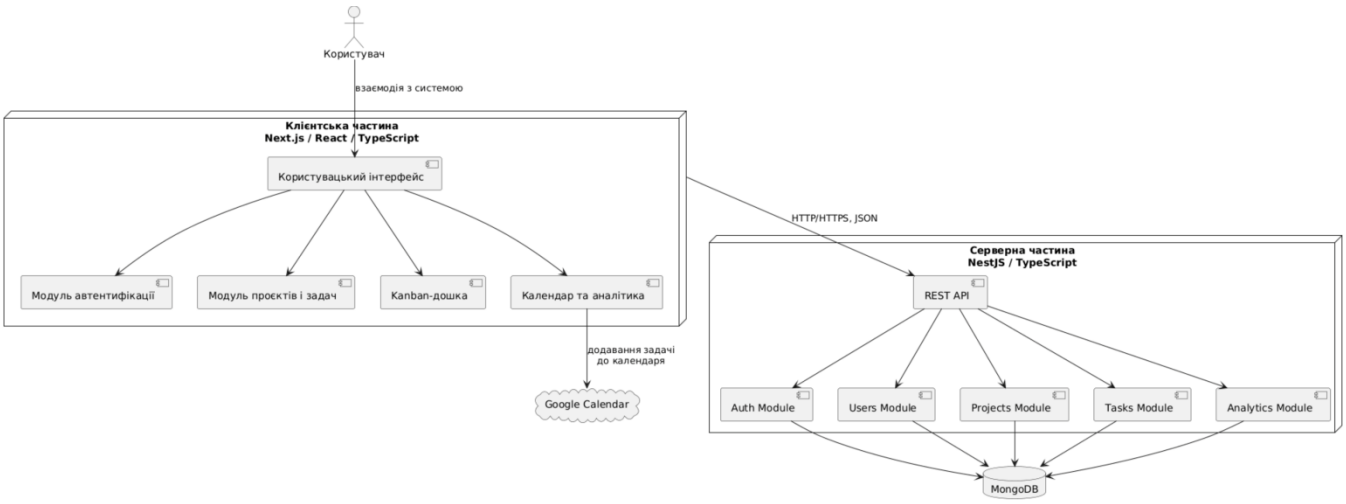


Рисунок В.1 – Діаграма компоненітв

В.4.2 Діаграма колекцій баз даних

Діаграма колекцій баз даних ілюструє логічну структуру MongoDB, що використовується у програмній системі управління проєктами. Вона відображає основні колекції, їхні зв’язки та спосіб організації даних для підтримки роботи з користувачами, проєктами й задачами.

База даних містить кілька основних колекцій. Колекція користувачів зберігає дані зареєстрованих користувачів, зокрема інформацію для автентифікації, особисті дані, ролі та налаштування профілю. Залежно від ролі користувач може виконувати функції адміністратора, менеджера проєкту або розробника. Колекція проєктів містить інформацію про створені проєкти, їх назву, опис, власника, учасників і службові дані. Проєкт є основною сутністю, яка об’єднує пов’язані задачі, колонки Kanban-дошки та учасників команди. Колекція задач зберігає інформацію про окремі задачі в межах проєкту. Кожна задача пов’язана з конкретним проєктом і колонкою, а також може містити назву, опис, виконавця, дедлайн, статус і мітки. Зв’язки між колекціями реалізуються за допомогою посилань MongoDB ObjectId. Користувачі пов’язані

з проєктами як власники або учасники, задачі належать до відповідних проєктів і колонок, а колонки формують структуру Kanban-дошки.

Діаграма колекцій бази даних демонструє (див. рис. В.2), як система організовує постійне зберігання даних і забезпечує взаємозв’язок між основними сутностями програмного забезпечення.

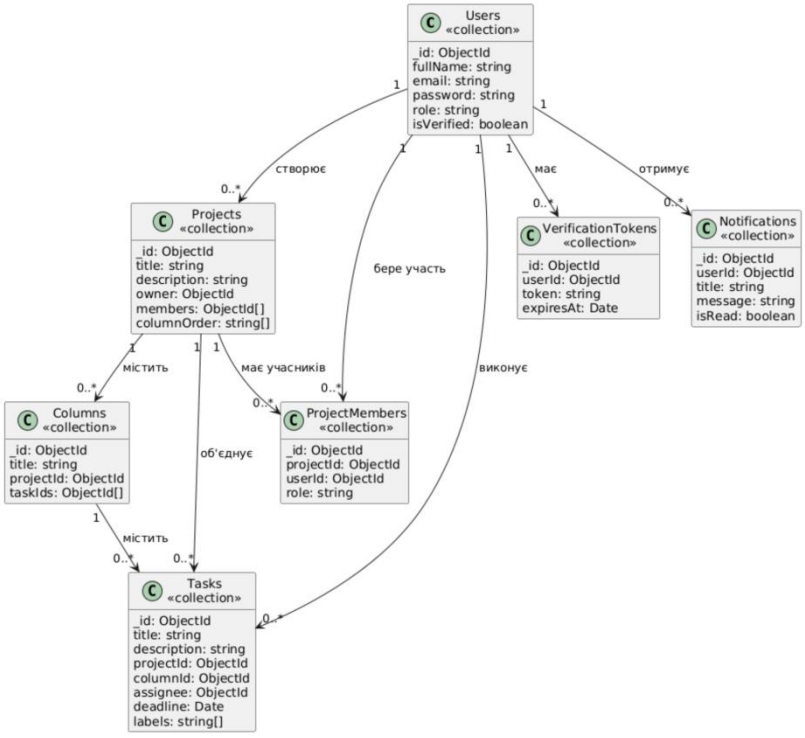


Рисунок В.2 – Діаграма колекцій бази даних

ДОДАТОК Г

Сценарії тестування програмної системи

Таблиця Г.1 – Основні сценарії тестування програмної системи

№	Модуль системи	Сценарій тестування	Очікуваний результат
1	Авторизація	Реєстрація нового користувача	Новий користувач успішно зареєстрований, запис збережено у базі даних
2	Авторизація	Вхід до системи	Користувач успішно авторизується та отримує доступ до системи
3	База даних	Перевірка збереження користувача	Документ користувача успішно створюється у колекції Users
4	Управління проєктами	Створення нового проєкту	Проєкт створено та відображено у списку проєктів
5	База даних	Перевірка збереження проєкту	Дані проєкту успішно збережено у колекції Projects
6	Управління завданнями	Створення нового завдання	Завдання успішно створено та відображено на Kanban-дошці
7	База даних	Перевірка збереження завдання	Новий документ успішно додається до колекції Tasks
8	База даних	Оновлення інформації про завдання	Усі зміни коректно зберігаються у базі даних
9	База даних	Зміна статусу завдання	Новий статус завдання успішно оновлюється у базі даних

Кінець таблиці Г.1

10	База даних	Видалення завдання	Документ успішно видаляється з колекції Tasks
11	Управління командою	Додавання учасника до проєкту	Новий учасник успішно додається до проєкту
12	База даних	Перевірка зв'язків між сутностями	Завдання залишається пов'язаним із відповідним проєктом та виконавцем
13	Коментарі	Додавання коментаря до завдання	Коментар відображається у завданні
14	База даних	Перевірка збереження коментаря	Коментар успішно зберігається у базі даних та відображається після повторного завантаження сторінки
15	База даних	Завантаження даних після повторної авторизації	Система успішно отримує актуальні дані про проєкти, завдання та користувачів із бази даних

ДОДАТОК Д

Створення колекцій бази даних MongoDB

Д.1 Схема колекції User:

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { Document } from 'mongoose';
import { UserRole } from '../../common/enums/role.enum';
@Schema({
  timestamps: true,
  toJSON: {
    transform: (doc, ret) => {
      delete ret.password;
      delete ret.__v;
      return ret;
    },
  },
})
export class User {
  @Prop({ required: true, unique: true })
  email: string;
  @Prop({ required: true })
  password: string;
  @Prop({ required: true })
  fullName: string;
  @Prop({ default: '' })
  avatarUrl: string;
  @Prop({ required: true, enum: Object.values(UserRole), default:
UserRole.DEVELOPER })
  role: UserRole;
  @Prop({ default: false })
  isVerified: boolean;
  @Prop()
  verificationToken?: string; // Додаємо опційність для TypeScript
  @Prop({ default: Date.now })
  createdAt: Date;
  @Prop()
  lastLogin: Date;
}
```

```
export type UserDocument = User & Document;
export const UserSchema = SchemaFactory.createForClass(User);
```

Д.2 Схема колекції Project:

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { Document, Types } from 'mongoose';
import { User } from '../../users/schemas/user.schema';
import { UserRole } from '../../common/enums/role.enum';
export class ProjectMember {
  @Prop({ type: Types.ObjectId, ref: 'User', required: true })
  user: Types.ObjectId;

  @Prop({ required: true, enum: Object.values(UserRole), default:
UserRole.DEVELOPER })
  role: UserRole;
}
@Schema({
  timestamps: true,
  toJSON: {
    virtuals: true,
    versionKey: false,
    transform: (_doc, ret) => {
      ret.id = ret._id;
      delete ret._id;
    }
  }
})
export class Project {
  @Prop({ required: true })
  name: string;
  @Prop()
  description: string;
  @Prop({ type: Types.ObjectId, ref: 'User', required: true })
  owner: User;
  @Prop({ type: [{ user: { type: Types.ObjectId, ref: 'User' }, role:
String }], default: [] })
  members: ProjectMember[];
```

```
  @Prop({ type: [String], default: [] })
  columnOrder: string[];
  @Prop({ default: Date.now })
  createdAt: Date;
  @Prop({ default: Date.now })
  updatedAt: Date;
}
export type ProjectDocument = Project & Document;
export const ProjectSchema = SchemaFactory.createForClass(Project);
// Virtual for `id` to map `_id` to `id` in JSON responses
ProjectSchema.virtual('id').get(function () {
  return this._id.toHexString();
});
```

Д.3 Схема колекції Notification:

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { Document, Types } from 'mongoose';
import { User } from '../../users/schemas/user.schema';
import { Task } from '../../tasks/schemas/task.schema';
export enum NotificationType {
  DEADLINE_APPROACHING = 'deadline-approaching',
  DEADLINE_MISSED = 'deadline-missed',
  TASK_ASSIGNED = 'task-assigned',
  TASK_COMPLETED = 'task-completed',
}
@Schema({ timestamps: true })
export class Notification {
  @Prop({ type: Types.ObjectId, ref: 'User', required: true })
  userId: User;
  @Prop({ type: Types.ObjectId, ref: 'Task', required: true })
  taskId: Task;
  @Prop({
    enum: NotificationType,
    required: true,
  })
  type: NotificationType;
  @Prop({ required: true })
  message: string;
```

```
  @Prop({ default: false })
  isRead: boolean;
  @Prop({ default: Date.now })
  createdAt: Date;
}
export type NotificationDocument = Notification & Document;
export const NotificationSchema =
SchemaFactory.createForClass(Notification);
```

Д.4 Схема колекції Column:

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { Document, Types } from 'mongoose';
import { Project } from '../../projects/schemas/project.schema';
@Schema({ timestamps: true })
export class Column {
  @Prop({ required: true })
  title: string;
  @Prop({ type: Types.ObjectId, ref: 'Project', required: true })
  projectId: Project;
  @Prop({ type: [String], default: [] })
  taskIds: string[];
  @Prop({ default: Date.now })
  createdAt: Date;
  @Prop({ default: Date.now })
  updatedAt: Date;
}
export type ColumnDocument = Column & Document;
export const ColumnSchema = SchemaFactory.createForClass(Column);
```

Д.5 Схема колекції Task:

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { Document, Types } from 'mongoose';
import { Column } from '../../columns/schemas/column.schema';
import { Project } from '../../projects/schemas/project.schema';
import { User } from '../../users/schemas/user.schema';
@Schema({ timestamps: true })
export class Task {
```

```
  @Prop({ required: true })
  title: string;
  @Prop()
  description: string;
  @Prop({ default: 'todo' })
  status: string;
  @Prop({ type: Types.ObjectId, ref: 'Column', required: true })
  columnId: Column;
  @Prop({ type: Types.ObjectId, ref: 'Project', required: true })
  projectId: Project;
  @Prop({ type: Types.ObjectId, ref: 'User', default: null })
  assignee: User;
  @Prop()
  deadline: Date;
  @Prop({ type: [String], default: [] })
  labels: string[];
  @Prop({ default: Date.now })
  createdAt: Date;
  @Prop({ default: Date.now })
  updatedAt: Date;
}
export type TaskDocument = Task & Document;
export const TaskSchema = SchemaFactory.createForClass(Task);
```